

Week 4: Detecting and Analysing Network Traffic with Suricata

Prepared by Mallikah Sharp



Introduction- Suricata

Suricata is a high-performance Network IDS (Intrusion Detection System), IPS (Intrusion Prevention System), and Network Security Monitoring (NSM) tool. By following the steps below, a SOC analyst can install Suricata and create custom rules for analysing the network traffic.

1. Install Suricata and update package list. Log into the Kali Linux Virtual Machine Terminal. To update Kali run command "`sudo apt update`", then to install Suricata, run command "`sudo apt install suricata`".

```
(kali@kali)-[~]
└─$ sudo apt install suricata
Installing:
suricata

Installing dependencies:
isa-support      librtt-bus-pci25  librtt-ip-frag25  librtt-meter25    librtt-ring25    snort-rules-default
libfdt1          librtt-bus-vdev25 librtt-kvargs25   librtt-net-bond25  librtt-sched25   sse3-support
libhtp2          librtt-eal25      librtt-log25      librtt-net25       librtt-telemetry25 sse4.2-support
libhyperscan5    librtt-ethdev25   librtt-mbuf25     librtt-pci25       libxdp1           suricata-update
libnetfilter-log1 librtt-hash25     librtt-mempool25  librtt-rcu25       oinkmaster

Suggested packages:
snort | snort-pgsql | snort-mysql libtcmalloc-minimal4

Summary:
Upgrading: 0, Installing: 30, Removing: 0, Not Upgrading: 1073
Download size: 6,987 kB
Space needed: 32.1 MB / 63.9 GB available

Continue? [Y/n] y
Get:1 http://mirror.ourhost.az/kali kali-rolling/main amd64 isa-support amd64 26 [15.0 kB]
Get:2 http://mirror.ourhost.az/kali kali-rolling/main amd64 sse3-support amd64 26 [3,532 B]
Get:3 http://mirror.ourhost.az/kali kali-rolling/main amd64 sse4.2-support amd64 26 [3,492 B]
Get:4 http://mirror.ourhost.az/kali kali-rolling/main amd64 libhtp2 amd64 1:0.5.50-1 [72.9 kB]
Get:5 http://mirror.ourhost.az/kali kali-rolling/main amd64 libhyperscan5 amd64 5.4.2-3 [2,695 kB]
Get:6 http://http.kali.org/kali kali-rolling/main amd64 libnetfilter-log1 amd64 1.0.2-4+b1 [13.3 kB]
```

2. Create a custom rule. Run command "`sudo nano /var/lib/suricata/rules/local.rules`"

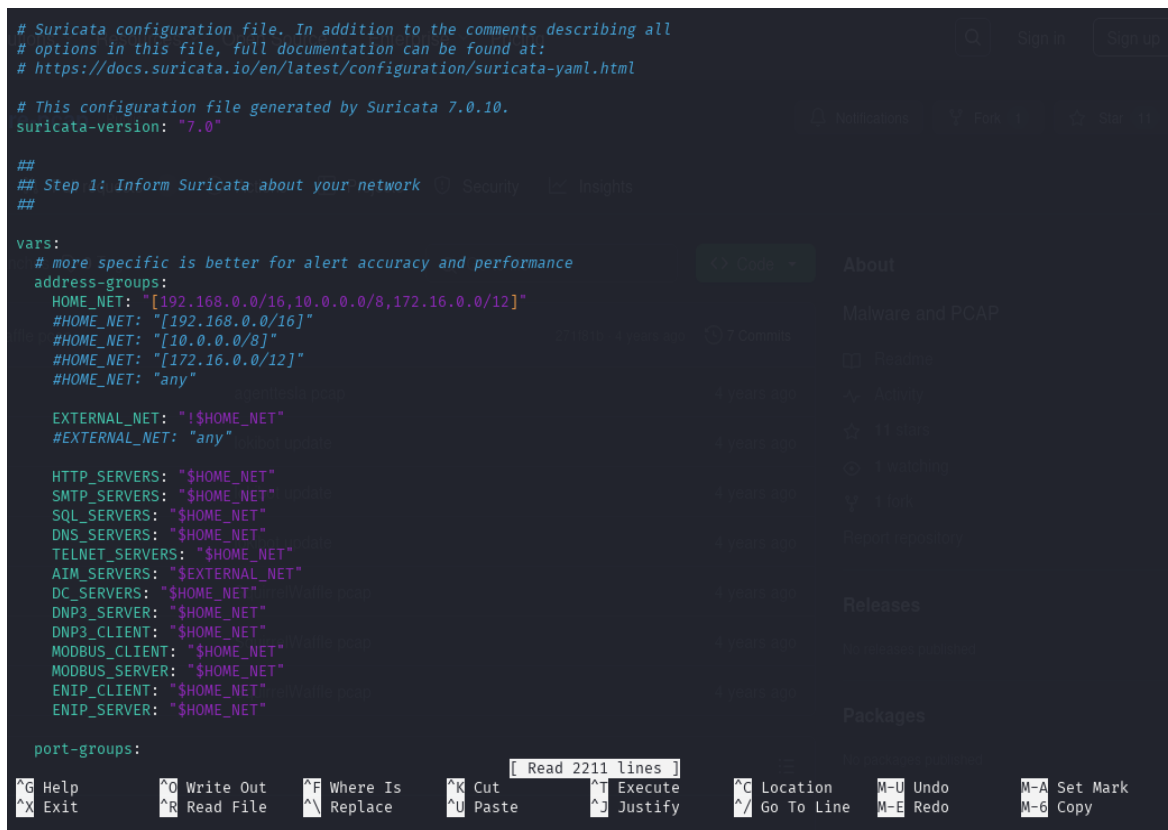
- **Add the rule to detect ICMP ping requests:** In the local.rules file, add a custom rule to detect ICMP ping requests. The rule format is based on Suricata's rule syntax.
- The following rule detects ICMP Echo Request (ping) packets:
Rule: `alert icmp any any -> any any (msg:"ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)`



```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.3 /var/lib/suricata/rules/local.rules *  
alert icmp any any -> any any (msg:"ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)  
|
```

- alert: Action to take when the rule matches.
- icmp: Protocol to match (ICMP).
- itype:8: ICMP type 8 corresponds to Echo Request (ping).
- sid:1000001: The unique rule identifier.
- rev:1: The revision number of the rule.
- msg: "ICMP Ping Detected": Message to display when the rule is triggered.

Then update Suricata configuration. Open the file by running the command “`sudo nano /etc/suricata/suricata.yaml`”.



```
# Suricata configuration file. In addition to the comments describing all  
# options in this file, full documentation can be found at:  
# https://docs.suricata.io/en/latest/configuration/suricata-yaml.html  
  
# This configuration file generated by Suricata 7.0.10.  
suricata-version: "7.0"  
  
##  
## Step 1: Inform Suricata about your network  
##  
  
vars:  
  # more specific is better for alert accuracy and performance  
  address-groups:  
    HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"  
    #HOME_NET: "[192.168.0.0/16]"  
    #HOME_NET: "[10.0.0.0/8]"  
    #HOME_NET: "[172.16.0.0/12]"  
    #HOME_NET: "any"  
  
    EXTERNAL_NET: "!$HOME_NET"  
    #EXTERNAL_NET: "any"  
  
    HTTP_SERVERS: "$HOME_NET"  
    SMTP_SERVERS: "$HOME_NET"  
    SQL_SERVERS: "$HOME_NET"  
    DNS_SERVERS: "$HOME_NET"  
    TELNET_SERVERS: "$HOME_NET"  
    AIM_SERVERS: "$EXTERNAL_NET"  
    DC_SERVERS: "$HOME_NET"  
    DNP3_SERVER: "$HOME_NET"  
    DNP3_CLIENT: "$HOME_NET"  
    MODBUS_CLIENT: "$HOME_NET"  
    MODBUS_SERVER: "$HOME_NET"  
    ENIP_CLIENT: "$HOME_NET"  
    ENIP_SERVER: "$HOME_NET"  
  
  port-groups:  
  
[ Read 2211 lines ]  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark  
^X Exit      ^R Read File  ^\ Replace    ^U Paste       ^J Justify    ^_ Go To Line  M-E Redo     M-G Copy
```

3. Next, Press Ctrl+f to set the “*default-rule*” path. This command takes you to the files Suricata uses for rules. Now include “*local.rules*” in the rule-files section list.

```
## Configure Suricata to load Suricata-Update managed rules.
##
default-rule-path: /var/lib/suricata/rules

rule-files:
- suricata.rules
- local.rules
##
## Auxiliary configuration files.
##

classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
# threshold-file: /etc/suricata/threshold.config

##
## Include other configs
##

# Includes: Files included here will be handled as if they were in-lined
# in this configuration file. Files with relative pathnames will be
# searched for in the same directory as this configuration file. You may
# use absolute pathnames too.
```

4. Apply the changes by restarting Suricata. Run "sudo systemctl restart suricata".
5. Then run Suricata to verify the rules are loaded "sudo suricata -c /etc/suricata/suricata.yaml -i eth0 -v"

```
(kali@kali)-[~]
$ sudo systemctl restart suricata

(kali@kali)-[~]
$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0 -v
Notice: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 2
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
```

6. Generate ICMP traffic to trigger the rule. In a different terminal, Run "ping -c 4 8.8.8.8"

```
(kali@kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=15.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=13.3 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=19.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=29.8 ms

— 8.8.8.8 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 13.273/19.491/29.794/6.377 ms
```

7. Verify the detection in the logs. Run "sudo cat /var/log/suricata/eve.json | grep "ICMP Ping Detected" (including the quotation marks)

*** If the rule is triggered, we should see an entry with the message "ICMP Ping Detected" in the log.

```
(kali@kali)-[~]
└─$ sudo cat /var/log/suricata/eve.json | grep "ICMP Ping Detected"
[sudo] password for kali:
{"timestamp": "2025-04-22T12:03:23.555531-0400", "flow_id": 978616188250588, "in_iface": "eth0", "event_type": "alert", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8", "proto": "ICMP", "icmp_type": 8, "icmp_code": 0, "pkt_src": "wire/pcap", "alert": {"action": "allowed", "gid": 1, "signature_id": 1000001, "rev": 1, "signature": "ICMP Ping Detected", "category": "", "severity": 3}, "direction": "to_server", "flow": {"pkts_to_server": 1, "pkts_to_client": 0, "bytes_to_server": 98, "bytes_to_client": 0, "start": "2025-04-22T12:03:23.555531-0400", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8"}}
{"timestamp": "2025-04-22T12:03:24.556706-0400", "flow_id": 978616188250588, "in_iface": "eth0", "event_type": "alert", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8", "proto": "ICMP", "icmp_type": 8, "icmp_code": 0, "pkt_src": "wire/pcap", "alert": {"action": "allowed", "gid": 1, "signature_id": 1000001, "rev": 1, "signature": "ICMP Ping Detected", "category": "", "severity": 3}, "direction": "to_server", "flow": {"pkts_to_server": 2, "pkts_to_client": 1, "bytes_to_server": 196, "bytes_to_client": 98, "start": "2025-04-22T12:03:23.555531-0400", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8"}}
{"timestamp": "2025-04-22T12:03:25.560472-0400", "flow_id": 978616188250588, "in_iface": "eth0", "event_type": "alert", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8", "proto": "ICMP", "icmp_type": 8, "icmp_code": 0, "pkt_src": "wire/pcap", "alert": {"action": "allowed", "gid": 1, "signature_id": 1000001, "rev": 1, "signature": "ICMP Ping Detected", "category": "", "severity": 3}, "direction": "to_server", "flow": {"pkts_to_server": 3, "pkts_to_client": 2, "bytes_to_server": 294, "bytes_to_client": 196, "start": "2025-04-22T12:03:23.555531-0400", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8"}}
{"timestamp": "2025-04-22T12:03:26.562965-0400", "flow_id": 978616188250588, "in_iface": "eth0", "event_type": "alert", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8", "proto": "ICMP", "icmp_type": 8, "icmp_code": 0, "pkt_src": "wire/pcap", "alert": {"action": "allowed", "gid": 1, "signature_id": 1000001, "rev": 1, "signature": "ICMP Ping Detected", "category": "", "severity": 3}, "direction": "to_server", "flow": {"pkts_to_server": 4, "pkts_to_client": 3, "bytes_to_server": 392, "bytes_to_client": 294, "start": "2025-04-22T12:03:23.555531-0400", "src_ip": "10.0.2.15", "dest_ip": "8.8.8.8"}}
```

***If you want to see other Suricata rules. Run the commands:

Step 1: `cd /var/lib/suricata/rules`

Step 2: `ls`

Step 3: `cat. suricata.rules`Then you will see all of the Snort rules.

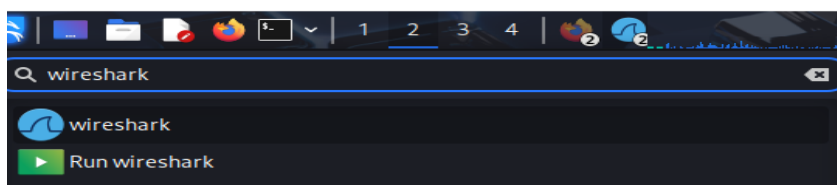
Wireshark



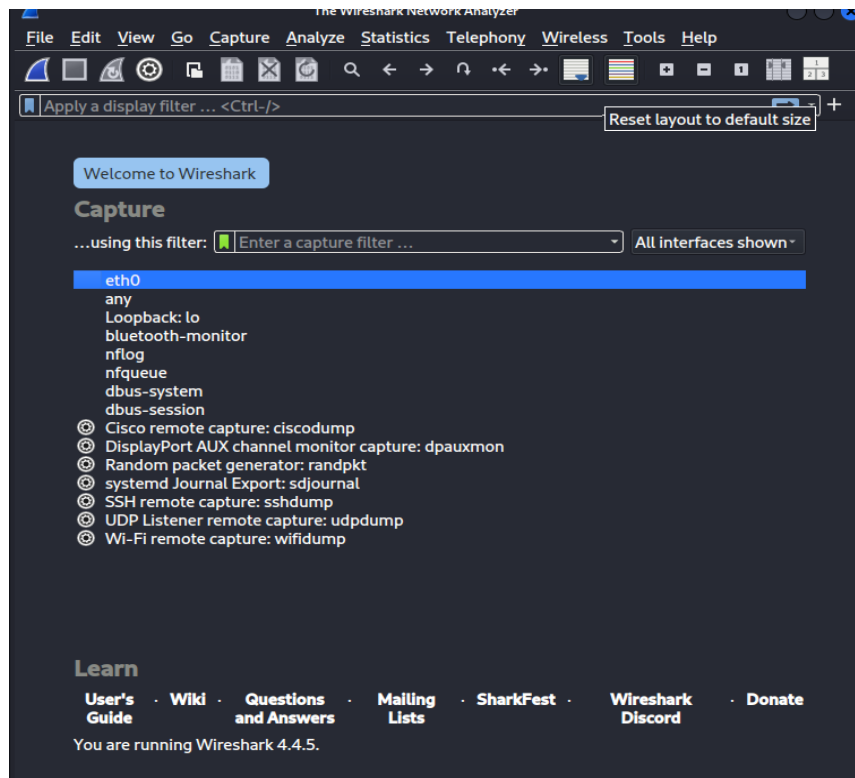
Introduction- Wireshark

Wireshark is a tool used to capture and analyse network traffic. Imagine it as a traffic camera for your network. It lets you see all the data being sent and received on your network in real-time.

1. To begin search for Wireshark in your Kali Linux VM by typing the name in the search bar.



2. A new window will pop up as Wireshark.



3. Then in your terminal, run command: "ifconfig"

```
(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::5ea3:6ea4:8221:ed16 prefixlen 64 scopeid 0x20<link>
    inet6 fd00::e21c:d0b0:44f9:7ad6 prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:04:42:0f txqueuelen 1000 (Ethernet)
    RX packets 1519581 bytes 2207083765 (2.0 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 91813 bytes 6014527 (5.7 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

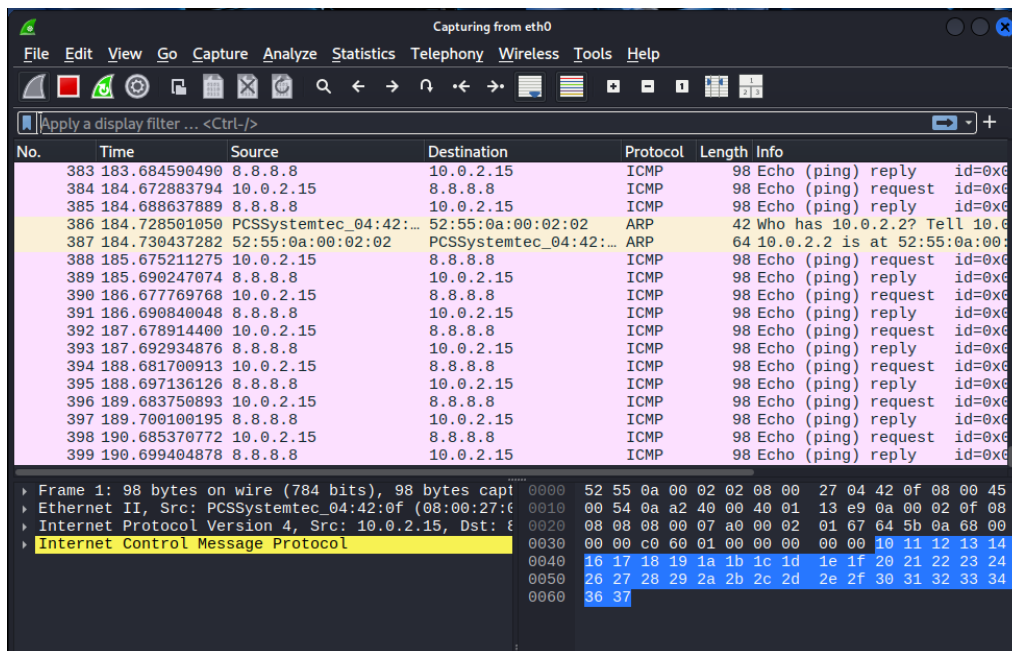
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 10 bytes 580 (580.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 10 bytes 580 (580.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

4. In the Kali terminal run command: "ping -c 4 8.8.8.8"
This is the Google public DNS server.

```
(kali㉿kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=14.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=12.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=14.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=12.9 ms

— 8.8.8.8 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 12.446/13.694/14.869/1.040 ms
```

- Then in the Wireshark window, press the blue shark fin in the upper left corner. This says "Start Running Packets" This will allow us to see the ICMP (Internet Control Message Protocol). To stop the packet capture collection, press the red square next to the blue fin.



You can check the details of each packet in the left lower window pane. You can also save the collection for later as a file on your machine.

Conclusion

This project helped me to understand Suricata's role in network monitoring and detection to act as an Intrusion Detection System (IDS). I was able to write and deploy custom detection rules to trigger alerts for ICMP ping attacks. With Wireshark, I learned how to install and set up live packet captures on a network. I understand how to filter traffic in Wireshark to focus on specific types of packets and inspect packet details, such as source and destination IPs, payloads and TTL (time-to-live). This project demonstrates my hands-on experience with IDS and traffic analysis tools.